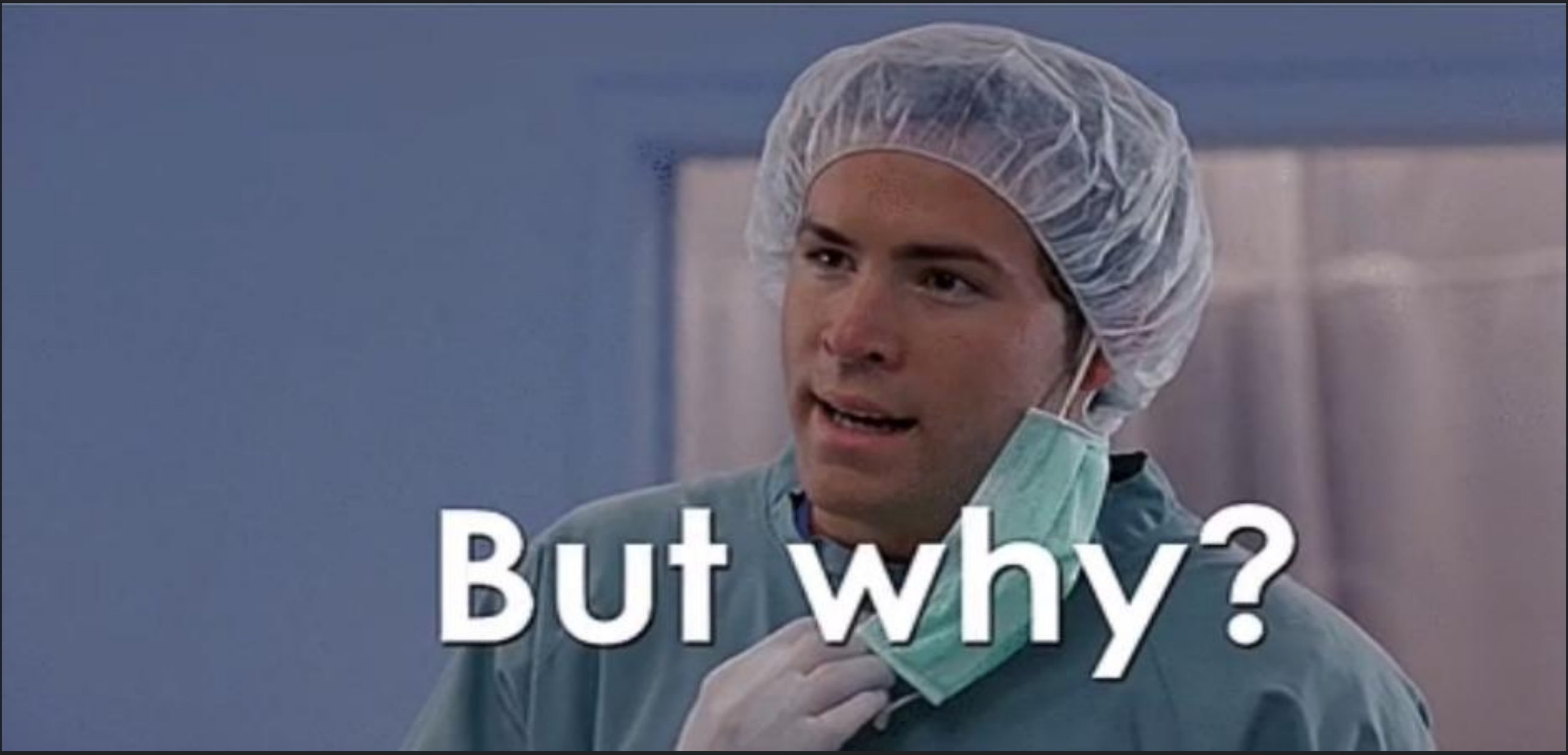
Google Developers

GDG

Persisting Data on Android Device storage

Marin Tolić
DICE


But why?

Reasons for Persisting Data

- Data is stored in-memory by default
- Devices go offline
- Per device user preferences

Flavors of Data

Unstructured or Loosely structured

- Primitives
- Small pieces of type safe structures
- No complex relationships

Unstructured or Loosely Structured Data

Unstructured or Loosely Structured

Available APIs

- SharedPreferences
- DataStore

Unstructured or Loosely Structured Data

SharedPreferences

Unstructured or Loosely Structured

Shared Preferences

- A simple API
- Allows encrypting data
- Not recommended by Google anymore

Using SharedPreferences

```
// Create an instance of shared preferences
val sharedPrefs = applicationContext.getSharedPreferences(
    // Preferences file name
    "preferences_file_key",
    // Operating mode
    Context.MODE_PRIVATE
)

val VERY_IMPORTANT_DATA_KEY = "very_important_data"

// Write to shared prefs
sharedPrefs.edit {
    putLong(VERY_IMPORTANT_DATA_KEY, 0L)
}

// Read from shared prefs
sharedPrefs.getLong(VERY_IMPORTANT_DATA_KEY, 0L)
```

Unstructured or Loosely Structured

Shared Preferences

- Clunky onChanged listeners
- Mostly suited to primitive data types
- No type safety

Unstructured or Loosely Structured Data

DataStore

Unstructured or Loosely Structured

Preferences DataStore

- Stores primitives
- No type safety
- Asynchronous data propagation

```kotlin
// Create preferences DataStore
val Context.dataStore: DataStore<Preferences> by
preferencesDataStore(name = "preferences_file_key")

// Define a key
val VERY_IMPORTANT_DATA =
longPreferencesKey("very_important_data")

// Write to preferences DataStore
suspend fun Context.writeImportantData() {
    this.dataStore.edit { settings ->
        settings[VERY_IMPORTANT_DATA] = 0L
    }
}


// Read from preferences DataStore
val exampleCounterFlow: Flow<Long> =
applicationContext.dataStore.data
    .map { preferences ->
        // No type safety.
        preferences[VERY_IMPORTANT_DATA] ?: 0L
    }
```

Unstructured or Loosely Structured

Proto DataStore

- Type safety
- Stores more complex data

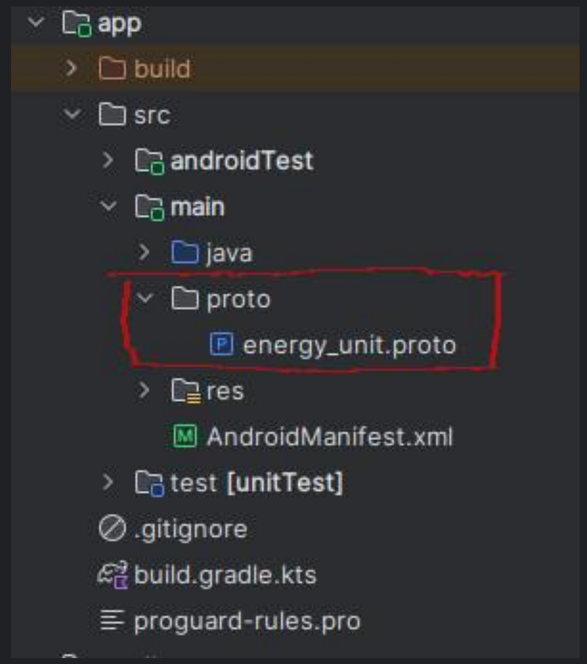
Setting up the Gradle Script

```
dependencies {
   implementation("com.google.protobuf:protobuf-javalite:4.29.3")
}

protobuf {
   protoc {
       libs.protobuf.protoc
       // Download from repositories
       artifact = "com.google.protobuf:protoc:4.29.3"
   }

   // Generates the java Protobuf-lite code for the Protobufs in this project. See
   // https://github.com/google/protobuf-gradle-plugin#customizing-protobuf-compilation
   // for more information.
   generateProtoTasks {
       all().forEach { task ->
           task.builtins {
               java {
                  id("java"){
                      option("lite")
                  }
               }
           }
       }
   }
}
```

Just a little bit more setting up!



Defining a proto file

```
syntax = "proto3";

option java_package =
"com.example.application.proto";
option java_multiple_files = true;

message EnergyUnit {
 string name = 1;
 double joulesConversionRatio = 2;
 message OrderOfMagnitude {
   string prefix = 3;
   int32 orderOfMagnitude = 4;
 }
 OrderOfMagnitude orderOfMagnitude = 5;
}
```

```kotlin
/**
* Responsible for serializing and deserializing energy unit from their protobuf format.
*/
object EnergyUnitSerializer : Serializer<EnergyUnit> {

    private val energyUnitBuilder = EnergyUnit.newBuilder()

    /**
     * The default value set in case the user has no stored data.
     */
    override val defaultValue: EnergyUnit = energyUnitBuilder…

    /**
     * Reads proto objects and returns their Java counterpart.
     */
    override suspend fun readFrom(input: InputStream): EnergyUnit {
        …
    }

    /**
     * Writes a proto object to storage from its Java representative.
     */
    override suspend fun writeTo(
        t: EnergyUnit,
        output: OutputStream
    ) = t.writeTo(output)
}
```

Data operations, finally!

```
// Creating a data store instance used for handling energy units
val Context.energyUnitDataStore: DataStore<EnergyUnit> by dataStore(
    fileName = "energy_unit.proto",
    serializer = EnergyUnitSerializer
)


// Write to DataStore
suspend fun Context.updateStoredEnergyUnits(energyUnit: com.example.habits_android.model.EnergyUnit) {
    this.energyUnitDataStore.updateData {
        it.toBuilder()
            .setName(energyUnit.name)
            …
            ).build()
    }
}

// Read from DataStore
fun Context.readStoredEnergyUnit() =
    this.energyUnitDataStore.data
        .map {
            com.example.habits_android.model.EnergyUnit(
                name = it.name,
                joulesConversionRatio = it.joulesConversionRatio,
                …
                )
            )
        }
```

Structured Data

Flavors of Data

Structured Data

- Complex relationships between objects
- Able to perform operations

Structured Data

Available APIs

- Room
- SQLDelight
- (R.I.P) Realm - Biggest refactoring hits since 2008

Structured Data

Room

Structured Data

Room

- First party
- SQL
- (Almost!) Multiplatform
- No encryption :(

Create your Entities

```
@Entity
data class MealEntity(
    @PrimaryKey val uuid: String = UUID.randomUUID().toString(),
    @ColumnInfo(name = "name") val name: String,
    @ColumnInfo(name = "weight") val weight: Int,
    @ColumnInfo(name = "averageEnergy") val averageEnergy: Int,
    @ColumnInfo(name = "averageProtein") val averageProtein: Int,
    @ColumnInfo(name = "averageCarbohydrates") val averageCarbohydrates: Int,
    @ColumnInfo(name = "averageFat") val averageFat: Int
)
```

DAO

```
@Dao
interface MealDao {
    @Query("SELECT * FROM mealentity")
    fun getAll(): Flow<List<MealEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(vararg mealEntities: MealEntity)
}
```

Define the Database

```
@Database(entities = [MealEntity::class], version = 1)
abstract class MealDatabase: RoomDatabase() {
    abstract fun mealDao(): MealDao
}
```

Finally, Instantiate Your Database

```
private val mealsDatabase by lazy {
    Room.databaseBuilder(
        applicationContext,
        MealDatabase::class.java, "meal-database"
    ).build()
}
```

Structured Data

Single Source of Truth

Make the Puzzle Pieces Fit

```
class MealRepository(
    // Fetches from the network
    private val mealsService: MealsService,
    private val mealDatabase: MealDatabase
) {
    // Fetch data from the server and store the response into the database
    suspend fun syncDatabase(): Result<Unit> = kotlin.runCatching {
        val response = mealsService.getAllMeals()

        if (response.isFailure) {
            return response.map { Unit }
        } else {
            val meals = response.getOrThrow()
                .map { it.toMealEntity() }

            mealDatabase.mealDao().insertAll(*meals.toTypedArray())
        }
    }

    // Fetch all the data from the database
    fun getAllMeals(): Flow<List<MealResponse>> = mealDatabase.mealDao().getAll()
        .map { list ->
            list.map { it.toMealResponse() }
        }

    // Add meals  to the server
    suspend fun addMeal(mealRequest: MealRequest) = mealsService.addMeal(mealRequest)
}
```

GitHub Repos

Android: https://github.com/MarinTolic/habits-android
Backend: https://github.com/MarinTolic/habits

GDG

Goodbye and thanks for all the fish